

lecture__06

August 1, 2026

1 Lecture 6: Python Scipy Method

Learning Objectives

By the end of this notebook, you will be able to: 1. Translate an LP formulation into the standard form required by `scipy.optimize.linprog` 2. Solve LP problems programmatically and extract the optimal solution and objective value.

Prerequisites: LP formulation (Lecture 3), Graphical Method (Lecture 4), Excel Solver (Lecture 5)

Estimated time: 90 minutes (including in-class exercises)

Environment check: Run the setup cell below before proceeding. This notebook requires `numpy`, `scipy`, and `matplotlib` — all available in standard Anaconda / Google Colab environments.

1.1 Why use Python SciPy?

In Lecture 5, we solved linear programs using the Excel Solver — a point-and-click tool well suited to quick, one-off analyses. Python's `scipy.optimize` module offers the same underlying solver capability but within a programmable environment, which unlocks several advantages that matter in practice:

Propoerty	Excel Solver	Python SciPy
Parametric analysis	Re-run Solver manually for each scenario	Loop over hundreds of scenarios in seconds
Reproducibility	Workbook with manual steps	Script that runs identically every time
Scale	Limited by spreadsheet size	Limited only by available memory
Visualisation	Basic Excel charts	Full <code>matplotlib</code> capability
Integration	Standalone file	Connects to databases, APIs, other models

For a one-time decision with a small model, Excel Solver is perfectly adequate. For research, recurring analyses, or problems with more than a handful of variables, Python is the right tool.

1.2 Setting up Python

Google Colab runs entirely in your browser — no installation required.

1. Go to colab.research.google.com
2. Click File → Upload notebook and upload this .ipynb file or create your own
3. All required libraries (numpy, scipy, etc.) are pre-installed — run the setup cell and you are ready

Tip: Go to Runtime → Change runtime type and select Python 3 if not already set. To save your work, click File → Save a copy in Drive.

```
[11]: # --- Environment Setup ---
import scipy                                # For linear programming
import numpy as np                          # For numerical operations

print(f"numpy version : {np.__version__}")
print(f"scipy version : {scipy.__version__}")
print("Setup complete.")
```

```
numpy version : 2.4.6
scipy version : 1.17.1
Setup complete.
```

1.3 The scipy.optimize.linprog Interface

linprog solves linear programs in the following **standard form**:

$$\min_{\mathbf{x}} \quad \mathbf{c}^\top \mathbf{x}$$

$$\text{subject to} \quad A_{ub} \mathbf{x} \leq \mathbf{b}_{ub}$$

$$A_{eq} \mathbf{x} = \mathbf{b}_{eq}$$

$$\mathbf{l} \leq \mathbf{x} \leq \mathbf{u}$$

1.3.1 Key Conversion Rules

Your LP	linprog equivalent
$\max \mathbf{c}^\top \mathbf{x}$	$\min -\mathbf{c}^\top \mathbf{x}$ (negate $\mathbf{c}$; negate result)
$a^\top x \geq b$	$-a^\top x \leq -b$ (negate both sides; add to A_{ub} , b_{ub})
$a^\top x = b$	Add to A_{eq} , b_{eq}
$x_j \geq 0$	Default: set <code>bounds=[(0, None), ...]</code>
x_j free	Set <code>bounds=[(None, None)]</code> for that variable

1.3.2 Function Signature

```
result = scipy.optimize.linprog(  
    c,                # Objective coefficients (1D array)  
    A_ub=None,        # Inequality LHS matrix (m × n)  
    b_ub=None,        # Inequality RHS vector (m,)  
    A_eq=None,        # Equality LHS matrix (p × n)  
    b_eq=None,        # Equality RHS vector (p,)  
    bounds=None,      # List of (lower, upper) per variable  
    method='highs'     # Always use 'highs' - fastest, most robust  
)
```

1.3.3 Result Object

Attribute	Meaning
<code>result.x</code>	Optimal decision variable values $\mathbf{x}^*$
<code>result.fun</code>	Optimal objective value (minimization)
<code>result.success</code>	True if optimal solution found
<code>result.message</code>	Status message

1.4 In-Class Exercises

1.4.1 Exercise 1

Same as Lecture 5 In-Class Exercise 1 — now solved in Python with richer analysis

A logistics company operates a Chennai–Delhi freight corridor. It can ship cargo either by truck or by rail. The planning horizon is one week.

	Truck	Rail
Revenue (/tonne)	4,500	3,200
Operating cost (/tonne)	2,800	1,400
CO emissions (kg/tonne)	62	18

Constraints:

1. Total capacity: at most 500 tonnes can be shipped in a week (combined)
2. Rail capacity: at most 300 tonnes by rail (track allocation limit)
3. Minimum service: at least 100 tonnes must go by truck (contractual obligation)
4. Emission cap: total CO must not exceed 24,000 kg
5. Non-negativity: tonnes shipped ≥ 0

Maximize total profit (revenue – operating cost).

Objective:

$$\max Z = 1700x_T + 1800x_R$$

Subject to:

$$\begin{aligned}x_T + x_R &\leq 500 && \text{(Capacity)} \\x_R &\leq 300 && \text{(Rail cap)} \\x_T &\geq 100 && \text{(Min truck)} \\62x_T + 18x_R &\leq 24000 && \text{(Emission cap)} \\x_T, x_R &\geq 0\end{aligned}$$

```
[ ]: # --- Exercise 1: Freight Mode Allocation ---

# Objective: maximize 1700*xT + 1800*xR => minimize -1700*xT - 1800*xR
c = np.array([-1700.0, -1800.0])

# Constraints: A_ub @ x <= b_ub
# [1] xT + xR <= 500 (total capacity)
# [2] xR <= 300 (rail cap)
# [3] -xT <= -100 (min truck: xT >= 100)
# [4] 62xT+18xR <= 24000 (emission cap)
A_ub = np.array([
    [ 1,  1],
    [ 0,  1],
    [-1,  0],
    [62, 18]
], dtype=float)
b_ub = np.array([500, 300, -100, 24000], dtype=float)

# Bounds: xT >= 0, xR >= 0
bounds = [(0, None), (0, None)]

# Solve the LP
result = scipy.optimize.linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds,
    ↪method='highs')

# Extract Results
xT, xR = result.x
profit = -result.fun

# Display Results
print("=" * 55)
print(" Exercise 1: Freight Mode Allocation")
print("=" * 55)
print(f" Status           : {result.message}")
print(f" Truck (xT)        : {xT:.1f} tonnes")
print(f" Rail (xR)          : {xR:.1f} tonnes")
print(f" Total Profit       : {profit:,.0f}")
print()
```

```

constraint_names = ['Total capacity', 'Rail capacity', 'Min truck (negated)', 'Emission cap']
rhs = [500, 300, -100, 24000]
lhs_vals = A_ub @ result.x

print(f"  {'Constraint':<25} {'LHS':>8} {'RHS':>8} {'Slack':>8}")
print("  " + "-" * 67)
for name, lhs, rhs_val in zip(constraint_names, lhs_vals, rhs):
    slack = rhs_val - lhs
    binding = " Binding" if abs(slack) < 1e-6 else ""
    print(f"  {name:<25} {lhs:>8.1f} {rhs_val:>8.1f} {slack:>8.1f} {binding}")

```

```
=====
```

Exercise 1: Freight Mode Allocation

```

Status          : Optimization terminated successfully. (HiGHS Status 7:
Optimal)
Truck (xT)      : 200.0 tonnes
Rail (xR)       : 300.0 tonnes
Total Profit    : 880,000

```

Constraint	LHS	RHS	Slack	
Total capacity	500.0	500.0	0.0	Binding
Rail capacity	300.0	300.0	0.0	Binding
Min truck (negated)	-200.0	-100.0	100.0	
Emission cap	17800.0	24000.0	6200.0	

1.4.2 Exercise 2

Same as Lecture 5 In-Class Exercise 2 — now solved in Python with richer analysis

A signalised intersection in Kolkata has 4 phases. The traffic engineer wants to allocate green time (in seconds) across phases to maximize total vehicle throughput (vehicles/cycle).

Phase	Movement	Saturation flow (veh/s)	Min green (s)	Max green (s)
1	Northbound through	0.52	10	45
2	Southbound through	0.48	10	45
3	Eastbound through	0.35	8	35
4	Westbound through	0.30	8	35

Constraints:

1. Cycle length = 90 seconds (fixed): $g_1 + g_2 + g_3 + g_4 = 90$

(we subtract a fixed lost time of 2s/phase $\rightarrow$ effective green budget = $90 - 4 \times 2 = 82s$)

2. Minimum green: $g_i \geq g_i^{\min}$ for each phase i

3. Maximum green: $g_i \leq g_i^{\max}$ for each phase i

Maximize total throughput = $\sum_i s_i \cdot g_i$ where s_i is saturation flow.

Objective:

$$\max Z = 0.52 g_1 + 0.48 g_2 + 0.35 g_3 + 0.30 g_4$$

Subject to:

$$g_1 + g_2 + g_3 + g_4 = 82 \quad (\text{Cycle budget})$$

$$g_i \geq g_i^{\min}, \quad i = 1, 2, 3, 4 \quad (\text{Min green})$$

$$g_i \leq g_i^{\max}, \quad i = 1, 2, 3, 4 \quad (\text{Max green})$$

```
[ ]: # --- Exercise 2: Signal Timing Optimization ---

# Objective: maximize 0.52*g1 + 0.48*g2 + 0.35*g3 + 0.30*g4 => minimize
# negative throughput
c = np.array([-0.52, -0.48, -0.35, -0.30])

# Constraints: A_eq @ x = b_eq
# [1] g1 + g2 + g3 + g4 = 82 (cycle budget: 90s - 4x2s lost time)
A_eq = np.ones((1, 4))
b_eq = np.array([82.0])

# Bounds: g_min <= g_i <= g_max
# Ph1: [10, 45], Ph2: [10, 45], Ph3: [8, 35], Ph4: [8, 35]
bounds = [(10, 45), (10, 45), (8, 35), (8, 35)]

# Solve the LP
result = scipy.optimize.linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
# method='highs')

# Extract Results
g1, g2, g3, g4 = result.x
throughput = -result.fun

# Display Results
print("=" * 55)
print(" Exercise 2: Signal Timing Optimization")
print("=" * 55)
print(f" Status : {result.message}")
```

```

print(f" Phase 1 (NB)      : {g1:.1f} s")
print(f" Phase 2 (SB)      : {g2:.1f} s")
print(f" Phase 3 (EB)      : {g3:.1f} s")
print(f" Phase 4 (WB)      : {g4:.1f} s")
print(f" Throughput        : {throughput:.3f} veh/cycle")
print()

constraint_names = ['Cycle budget (=)',
                    'Ph 1 lower bound', 'Ph 2 lower bound', 'Ph 3 lower bound',
                    ↪ 'Ph 4 lower bound',
                    'Ph 1 upper bound', 'Ph 2 upper bound', 'Ph 3 upper bound',
                    ↪ 'Ph 4 upper bound']
lhs_vals = [sum(result.x)] + list(result.x) + list(result.x)
rhs_vals  = [82.0, 10, 10, 8, 8, 45, 45, 35, 35]

print(f" {'Constraint':<22} {'LHS':>8} {'RHS':>8} {'Slack':>8}")
print(" " + "-" * 55)
for name, lhs, rhs_val in zip(constraint_names, lhs_vals, rhs_vals):
    slack = rhs_val - lhs
    binding = " Binding" if abs(slack) < 1e-6 else ""
    print(f" {name:<22} {lhs:>8.1f} {rhs_val:>8.1f} {slack:>8.1f} {binding}")

```

Exercise 2: Signal Timing Optimization

Status : Optimization terminated successfully. (HiGHS Status 7: Optimal)

Phase 1 (NB) : 45.0 s
Phase 2 (SB) : 21.0 s
Phase 3 (EB) : 8.0 s
Phase 4 (WB) : 8.0 s
Throughput : 38.680 veh/cycle

Constraint	LHS	RHS	Slack	
Cycle budget (=)	82.0	82.0	0.0	Binding
Ph 1 lower bound	45.0	10.0	-35.0	
Ph 2 lower bound	21.0	10.0	-11.0	
Ph 3 lower bound	8.0	8.0	0.0	Binding
Ph 4 lower bound	8.0	8.0	0.0	Binding
Ph 1 upper bound	45.0	45.0	0.0	Binding
Ph 2 upper bound	21.0	45.0	24.0	
Ph 3 upper bound	8.0	35.0	27.0	
Ph 4 upper bound	8.0	35.0	27.0	

1.5 Take-Away Exercises

1.5.1 Exercise 1

Problem Statement *Same as Lecture 5 Take-Away Exercise 1 — now solved in Python*

The Chennai Metropolitan Transport Corporation (MTC) operates two routes: - Route A (high-demand urban corridor): requires at least 3,200 passenger-km/day - Route B (suburban feeder): requires at least 1,800 passenger-km/day

MTC can deploy two bus types:

	Standard Bus	Articulated Bus
Capacity (pass-km/day per bus)	400	750
Operating cost (/day per bus)	8,500	14,000
Maintenance cost (/day per bus)	1,200	2,500
Driver requirement (drivers/bus)	1	2

MTC must satisfy five operational constraints. On the demand side, Route A must deliver at least 3,200 pass-km/day and Route B at least 1,800 pass-km/day. On the supply side, the daily operating budget cannot exceed 2,00,000, the driver pool is limited to 60, and the total fleet to 25 buses.

Assume each bus type can be split between routes. Let: - x_1 = standard buses deployed (total across both routes) - x_2 = articulated buses deployed (total across both routes)

Minimize total daily cost.

Your Tasks A. Solve the LP using `scipy.optimize.linprog`.

B. Infer the Results: - Report the optimal fleet mix and cost. - Identify binding and non-binding constraints.

C. Add a CO₂ constraint: standard buses emit 1.8 kg CO₂/pass-km, articulated emit 1.2 kg CO₂/pass-km. Total daily CO₂ 12,000 kg. Does this change the optimal solution? By how much does cost increase?

```
[ ]: import scipy
import numpy as np

# Problem data
cost = np.array([9700, 16500])          # objective coefficients
cap = np.array([400, 750])              # capacity per bus type (pass-km/day)

# Objective

# Constraints

# Bounds

# Solve the LP
```

```
# Extract Results
```

```
# Present Results
```

1.5.2 Exercise 2

Problem Statement A state transport depot in Delhi maintains a fleet of 200 buses and needs to blend two grades of diesel to meet fleet specifications:

Specification	Fleet Requirement
Cetane number	48
Sulphur content	15 ppm
Density	Between 820 and 845 kg/m ³

Two diesel grades are available:

Property	Grade A (High-quality)	Grade B (Standard)
Cetane number	55	42
Sulphur (ppm)	8	22
Density (kg/m ³)	830	850
Cost (/litre)	92	79

The depot needs 10,000 litres/day total. Blended properties are volume-weighted averages.

Let: - x_A = litres of Grade A per day - x_B = litres of Grade B per day

Minimize daily fuel cost.

Your Tasks A. Solve using `scipy.optimize.linprog`.

B. Infer the results - Report optimal blend (litres and percentages) and daily cost. - Identify binding and non-binding constraints.

C. A third grade, Grade C (85/L), becomes available: cetane = 50, sulphur = 12 ppm, density = 838 kg/m³. Add x_C to the model and re-solve. Does the depot use Grade C? Why or why not?

```
[ ]: import scipy
import numpy as np

# Problem data
cost_A, cost_B = 92.0, 79.0
cetane_A, cetane_B = 55.0, 42.0
sulphur_A, sulphur_B = 8.0, 22.0
density_A, density_B = 830.0, 850.0

# Objective
```

```

# Constraints

# Bounds

# Solve the LP

# Extract Results

# Present Results

```

1.5.3 Exercise 3

Problem Statement *Same as Lecture 5 Take-Away Exercise 3 — now solved in Python with richer analysis*

The Kerala State Highway Authority has a budget of 90 crore to widen single-lane stretches of state highway to two lanes. There are 5 candidate stretches identified by the annual road-safety audit. Widening a stretch reduces its annual accident count, based on engineering studies of crash rates on single- vs. two-lane sections of comparable traffic volume.

Stretch	Widening Cost (Cr)	Annual Accidents Prevented	Length (km)
S1 — Kochi– Aluva	32	18	14
S2 — Thrissur– Guruvayur	24	11	9
S3 — Kollam– Punalur	28	14	12
S4 — Kottayam– Erattupetta	18	8	7
S5 — Kannur– Thalassery	22	12	10

Maximise total accidents prevented per year by deciding what fraction $y_s \in [0, 1]$ of each stretch to widen (a partially-funded stretch widens a proportional length, preventing a proportional share of accidents — a standard assumption in multi-year phased capital planning).

Your Tasks A. Solve using `scipy.optimize.linprog`.

B. Infer the results - Report optimal y_i^* . Identify fully upgraded, partially upgraded, and non-upgraded links. - Identify binding and non-binding constraints.

C. If the government values travel time at 200/person-hour, what is the monetary value of the optimal investment at 120 Cr budget? What is the benefit-cost ratio?

D. Round the LP solution to nearest integer (0 or 1). Is the rounded solution feasible? Compare its total benefit to the LP relaxation.

```
[ ]: import scipy
import numpy as np

# Stretch data
costs = np.array([32, 24, 28, 18, 22])          # Crore
accidents_prevented = np.array([18, 11, 14, 8, 12]) # per year, at full widening

# Objective

# Constraints

# Bounds

# Solve the LP

# Extract Results

# Present Results
```

1.6 Excel Solver vs Python `linprog` — When to Use Which?

Criterion	Excel Solver	Python <code>linprog</code>
Setup time	Fast for one-off problems	More setup; pays off for repeated use
Parametric analysis	Requires re-running manually	Loop in seconds; unlimited scenarios
Problem size	Limited (Excel cell range)	Limited only by RAM
Reproducibility	Workbook (hard to version)	Script in Git (fully reproducible)
Visualisation	Basic Excel charts	Full <code>matplotlib</code> / <code>seaborn</code> capability
Integration	Standalone workbook	Integrates with data pipelines, databases, APIs
Audience	Managers, planners (non-programmers)	Engineers, researchers, data scientists

Practical advice: Use Excel Solver for quick feasibility checks, stakeholder presentations, and one-time analyses. Use Python for production models, sensitivity studies, and research — where reproducibility, scale, and automation matter.

1.7 Further Reading

- SciPy documentation: `scipy.optimize.linprog` — [docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize.linprog.html)
- Vanderbei, R.J. (2014). *Linear Programming: Foundations and Extensions* — theoretical background on the HiGHS simplex

- Transportation Research Board (2010). *Highway Capacity Manual* — signal timing optimization (Chapter 19)